INDEX

PART - A

EX. NO.	CONTENTS	PAGE NO.	REMARKS
1	**Create a table EMPLOYEE using SQL command to store details of employees such as EMPNO, NAME, DESIGNATION, DEPARTMENT, GENDER and SALARY. Specify Primary Key and NOT NULL constraints on the table. Allow only 'M' or 'F' for the column GENDER. DEPARTMENT can be SALES, ACCOUNTS, IT. Choose DESIGNATION as CLERK, ANALYST, MANAGER, ACCOUNTANT and SUPERVISOR that depends on department Write the following SQL queries:** **a) Display EMPNO, NAME and DESIGNATION of all employees whose name ends with RAJ.** **b) Display the details of all female employees who is earning salary within the range 20000 to 40000 in SALES or IT departments** **c) List the different DEPARTMENTs with the DESIGNATIONs in that department.** **d) Display the department name, total, average, maximum, minimum salary of the DEPARTMENT only if the total salary given in that department is more than 30000.** **e) List the departments which have more than 2 employees.**		
2	**Create a table CLIENT to store CLIENT_NO, NAME, ADDRESS, STATE, BAL_DUE. Client no must start with 'C'. Apply the suitable structure for the columns. Specify Primary Key and NOT NULL constraints on the table. Insert 10 records. Write the following SQL queries:** **a) From the table CLIENT, create a new table CLIENT1 that contains only CLIENT_NO and NAME , BAL_DUE from specified STATE. Accept the state during run time.** **b) Create a new table CLIENT2 that has the same structure as CLIENT but with no records. Display the structure and records.** **c) Add a new column by name PENALTY number (10, 2) to the CLIENT** **d) Assign Penalty as 10% of BAL_DUE for the clients C1002, C1005, C1009 and for others 8%. Display Records** **e) Change the name of CLIENT1 as NEW_CLIENT f) Delete the table CLIENT2**		

3	**Create a table BOOK using SQL command to store Accession No, TITLE, AUTHOR, PUBLISHER, YEAR, PRICE. Apply the suitable structure for the columns. Specify Primary Key and NOT NULL constraints on the table. Insert 10 records. Write the following SQL queries:** **a) List the details of publishers having 'a' as the second character in their names.** **b) Display Accession No., TITLE, PUBLISHER and YEAR of the books published by the specified author before 2010 in the descending order of YEAR. Accept author during run time** **c) Modify the size of TITLE to increase the size 5 characters more.** **d) Display the details of all books other than Microsoft press publishers. e) Remove the records of the books published before 1990.**		
4	**Create a table SALES with columns SNO, SNAME, MNO, JOIN_DATE, DATE_BIRTH, SALARY, SALES_AMOUNT and COMMISSION. Minimum age for joining the company must be 18 Yrs. Default value for Commission should be 0. Apply the suitable structure for the columns. Specify Primary Key and NOT NULL constraints on the table. Insert 10 records with data except commission. Manager of Manager can be NULL. Write the following SQL queries:** **a) Display the details of Sales Persons whose salary is more than Average salary in the company.** **b) Update commission as 20% of Sales Amount.** **c) Display SNO, SNAME, MNO, SALARY, COMMISSION, MANAGER_SALARY of the sales persons getting sum of salary and commission more than salary of manager. (Self-join)** **d) Display the records of employees who finished the service of 10years**		
5	**Create a table Sales_Details with the columns SNO, MONTH, TARGET and QTY_SOLD to store the Sales Details of one year. Specify the composite primary key to the columns SNO and MONTH. TARGET and SALES must be positive numbers.** **Write the following SQL queries:** **a) Display the total sales by each sales person considering only those months sales where target was reached** **b) If a commission of RS.50 provided for each item after reaching target, calculate and display the total commission for each sales person.** **c) Display the SNO of those who never reached the target.** **d) Display the SNO, MONTH and QTY_SOLD of the sales persons with SNO S0001 or S0003**		
6	**Create the following tables by identifying primary and foreign keys. Specify the not null property for mandatory keys.** **SUPPLIERS (SUPPLIER_NO, SNAME, SADDRESS,SCITY)** **COMPUTER_ITEMS(ITEM_NO, SUPPLIER_NO,ITEM_NAME, IQUANTITY) Consider three suppliers. A supplier can supply more than one type of items. Write the SQL queries for the following:** **a)List ITEM and SUPPLIER details in alphabetical order of city name and in each city decreasing order of IQUANTITY.**		

b) List the name ,city,and address of the suppliers who are supplying keyboard.
c) List the supplier name, items supplied by the suppliers 'Cats' and 'Electrotech'.
d) Find the items having quantity less than 5 and insert the details of supplier and item of these, into another table NEWORDER

7 **Create the following tables by identifying primary and foreign keys, specify the not null property for mandatory keys.**

PRODUCT DETAIL				
P_NO	**PRODUCT NAME**	**QTY AVAILABLE**	**PRICE**	**PROFIT %**
P0001	**Monitor**	**10**	**3000**	**20**
P0002	**Pen Drives**	**50**	**650**	**5**
P0003	**CD Drive**	**100**	**10**	**3**
P0004	**Key Board**	**25**	**600**	**10**

PURCHASE DETAILS		
CUSTNO	**P_NO**	**QTYSOLD**
C1	**P00003**	**2**
C2	**P0002**	**4**
C3	**P0002**	**10**
C4	**P0001**	**3**
C1	**P0004**	**2**
C2	**P0003**	**2**
C4	**P0004**	**1**

Write the following SQL queries:
a) Display total amount spent by C2.
b) Display the names of product for which either QtyAvailable is less than 30 or total QtySold is less than 5(USE UNION).
c) Display the name of products and quantity purchased by C4.
d) How much Profit does the shopkeeper gets on C1's purchase?
e) How many 'Pen Drives' have been sold?

PART - B

EX. NO.	CONTENTS	PAGE NO.	REMARKS
1.	**Create a table Bank with the columns ACNO, ACT_NAME, ACT_TYPE and BAL. Specify the Primary Key. Initial BAL must be greater than 500.** **Write a PL/SQL program to perform debit operation by providing acct_no and amount required. The amount must be greater than 100 and less than 20000 for one transaction.** **If the account exist and BAL-amount>500 Bank table must be updated, otherwise "NO SUFFFICIENT BALANCE" message should be displayed. If account number is not present then display "NO SUCH ACCOUNT" message to the user.**		
2.	**Create a table STOCK_DETAIL with the columns PNO, PNAME and QTY_AVL to store stock details of computer accessories. Specify Primary Key and NOT NULL constraints on the table. QTY_AVL should be positive number. Write a PL/SQL Program to define a user defined exception named "LOW_STOCK" to validate the transaction. The program facilitates the user to purchase the product by providing product number and quantity required. It should display an error message "NO SUFFICIENT STOCK" when the user tries to purchase a product with quantity more than QTY_AVL, Otherwise the STOCK_DETAIL table should be updated for valid transaction.**		
3.	**Write a PL/SQL program to compute the selling price of books depending on the book code and category. Use Open, Fetch and Close.** **The Book detail table contains columns: Book Code, Author, Title, Category and Price.** **Insert 10 records.** **The selling price-Price-Discount.** **The discount is calculated as follows:**		

BOOK CODE	CATEGORY	DISCOUNT PERCENTAGE
A	Novels	10% of Price
	Technology	12.5% of Price
B	Commerce	18% of Price
	Science	19% of Price
C	Songs	25% of Price
	Sports	24% of Price
D	All	28% of Price

Print the result in tabular form with proper alignment

Book Code category title author price discount % discount amount sell price

========= ========== ===== ============ ======= ====== === ====

4. **Write a PL/SQL program to display employee pay bill (using Cursor For loop) Use a Procedure to receive basic pay and to compute DA, HRA, Tax, PF, Gross Pay and Net Pay (Use OUT). Base table contains the following columns empnum, empname, basic pay. Insert 3 records. Allowances are computed as follows.**

Basic Pay	DA	HRA
<=20000	35% of Basic	8% of Basic
>20000&<=30000	38%	9%
>30000&<=40000	40%	10%
>40000	45%	10%

Gross=Basic+DA+HRA
PF=12% of Gross or Rs. 2000 whichever is minimum.
PT=Rs. 100 upto Gross is 25,000 else Rs. 200.
Net=Gross-(PF+PT)
Print Pay slip as follows

```
----------PAYSLIP----------
Empno          :10011          Empname : Raj
Basic Pay      :20000            P.F.: 3432
DA             :7000             P.T.: 200
H.R.A.         :1600
Gross          :28600        Net Pay : 24968
==================================================
----------PAYSLIP----------
Empno          :10012          Empname : Rani
Basic Pay      :30000            P.F.: 5292
DA             :11400            P.T.: 200
H.R.A.         :2700
Gross          :44100        Net Pay : 38608
==================================================
```

5. **Given the following tables: ITEM_MASTER (item no, name, stock, unit price) [Apply the Primary key and check constraint for stock and price as >0] [Insert 5 records] ITEM_TRANS (item no, quantity and trans_date) Create a package PCK_ITEM includes a function CHK_ITEM and a procedure PROC_ITEM. Function CHK_ITEM gets one argument itemno and is used to check whether the parameter itemno exists in ITEM_MASTER and should return 1 if exist. Otherwise, 0 and**
displays proper message. Procedure PROC_ITEM gets two arguments itemno and quantity, and is used to perform the following if item exists. If required quantity is not available, give appropriate message. If available, insert a record of this transaction to ITEM_TRANS and modify the stock in ITEM_MASTER. Write a PL/SQL program to accept ITEM_NO and Quantity needed of required item. Use Package to do the transaction process (Transaction date can be current date). OUTPUT to be shown as follows:
Enter value for accept itemno: 1
old 5: X:=&accept itemno:
new 5: X:-1:
Enter value for quantity: 3
old 6: M:&quantity:
new 6: H: 3:
Item :aa Quantity :3 Price :15 Total Amount :45

6.	**Create a package which includes a function to compute the factorial of a number, a procedure to compute the value of nCr, and another procedure to compute nPr both uses the factorial function. Execute the package program for the required calculation.**		
7.	**Create a trigger to update the MASTER table when a record is inserted into SALES table and create another trigger to update the MASTER table when a record is inserted or updated or deleted in NEWSTOCK table. Assume the suitable columns for all the tables.**		

PART-A

Program 1:

Create a table EMPLOYEE using SQL command to store details of employees such as EMPNO, NAME, DESIGNATION, DEPARTMENT, GENDER and SALARY. Specify Primary Key and NOT NULL constraints on the table. Allow only 'M' or 'F' for the column GENDER. DEPARTMENT can be SALES, ACCOUNTS, IT. Choose DESIGNATION as CLERK, ANALYST, MANAGER, ACCOUNTANT and SUPERVISOR that depends on department Write the following SQL queries:

a) Display EMPNO, NAME and DESIGNATION of all employees whose name ends with RAJ.

b) Display the details of all female employees who is earning salary within the range 20000 to 40000 in SALES or IT departments

c) List the different DEPARTMENTs with the DESIGNATIONs in that department.

d) Display the department name, total, average, maximum, minimum salary of the DEPARTMENT only if the total salary given in that department is more than 30000.

e) List the departments which have more than 2 employees.

```
SQL> create table employee(
    empno varchar(10) primary key,
    name varchar(30) not null,
   designation varchar(20) not null check(designation
in('clerk','analyst','manager','accountant','supervisior')),
    department varchar(20) not null check(department in('SALES','ACCOUNTS','IT')),
   gender char(1) not null check(gender in('M','F')),
   salary number(10) not null);

Table created.
```

```
SQL> desc employee;
Name                                      Null?                Type
EMPNO                                     NOT NULL             VARCHAR2(10)
NAME                                      NOT NULL             VARCHAR2(30)
DESIGNATION                               NOT NULL             VARCHAR2(20)
DEPARTMENT                                NOT NULL             VARCHAR2(20)
GENDER                                    NOT NULL             CHAR(1)
SALARY                                    NOT NULL             NUMBER(10)

SQL> insert into employee values('A101','Ram','manager','IT','M',12000);
1 row created.
SQL> insert into employee values('A102','Dheeraj','analyst','IT','M',20000);
1 row created.
SQL> insert into employee values('A103','charanraj','clerk','SALES','M',40000);
1 row created.
SQL> insert into employee values('A104','kavya','supervisior','SALES','F',50000);
1 row created.
SQL> insert into employee values('A105','Rakshitha','analyst','IT','F',40000);
1 row created.
SQL> insert into employee values('A106','Ashmitha','clerk','SALES','F',20000);
1 row created.

SQL> select * from employee;
EMPNO    NAME      DESIGNATION      DEPARTMENT      G      SALARY
A101     Ram       manager          IT              M      12000
A102     Dheeraj   analyst          IT              M      20000
A103     charanraj clerk            SALES           M      40000
A104     kavya     supervisior      SALES           F      50000
A105     Rakshitha analyst          IT              F      40000
```

A106	Ashmitha	clerk	SALES	F	20000

6 rows selected.

a) SQL> select empno,name,designation from employee where name like'%raj';

EMPNO	NAME	DESIGNATION
A102	Dheeraj	analyst
A103	charanraj	clerk

b) SQL> select * from employee where gender='F' and salary between 20000 and 40000 and department in('SALES','IT');

EMPNO	NAME	DESIGNATION	DEPARTMENT	G	SALARY
A105	Rakshitha	analyst	IT	F	40000
A106	Ashmitha	clerk	SALES	F	20000

c) List the different DEPARTMENTs with the DESIGNATIONs in that department.

SQL> select distinct department ,designation from employee;

DEPARTMENT	DESIGNATION
SALES	clerk
SALES	supervisior
IT	manager
IT	analyst

d)Display the department name, total, average, maximum, minimum salary of the DEPARTMENT only if the total salary given in that department is more than 30000.

SQL> select department,sum(salary)as total_salary,avg(salary)as average_salary,max(salary)as max_salary,min(salary)as min_salary from employee group by department having sum(salary)>30000;

DEPARTMENT	TOTAL_SALARY	AVERAGE_SALARY	MAX_SALARY	MIN_SALARY
IT	72000	24000	40000	12000
SALES	110000	36666.6667	50000	20000

e) List the departments which have more than 2 employees.

SQL> select department from employee group by department having count (*)>2;

DEPARTMENT

IT

SALES

Program 2:

Create a table CLIENT to store CLIENT_NO, NAME, ADDRESS, STATE, BAL_DUE. Client no must start with 'C'. Apply the suitable structure for the columns. Specify Primary Key and NOT NULL constraints on the table. Insert 10 records. Write the following SQL queries:

a) From the table CLIENT, create a new table CLIENT1 that contains only CLIENT_NO and NAME, BAL_DUE from specified STATE. Accept the state during run time.

b) Create a new table CLIENT2 that has the same structure as CLIENT but with no records. Display the structure and records.

c) Add a new column by name PENALTY number (10, 2) to the CLIENT

d) Assign Penalty as 10% of BAL_DUE for the clients C1002, C1005, C1009 and for others 8%. Display Records

e) Change the name of CLIENT1 as NEW_CLIENT

f) Delete the table CLIENT2

```
SQL> create table client(
    client_no varchar(10) primary key check(client_no like 'C%'),
    name varchar(50) not null,
   address varchar(30),
    state varchar(30) not null,
    bal_due number(10,2) not null);

Table created.

SQL> desc client;
Name                      Null?          Type
 CLIENT_NO                NOT NULL       VARCHAR2(10)
 NAME                     NOT NULL       VARCHAR2(50)
 ADDRESS                                 VARCHAR2(30)
```

STATE	NOT NULL	VARCHAR2(30)
BAL_DUE	NOT NULL	NUMBER(10,2)

```
SQL> insert into client values('C1001','Alice','123 Park lane','Karnataka',1500.00);
1 row created.
SQL> insert into client values('C1002','Bob','456 Hill street','Kerala',2000.00);
1 row created.
SQL> insert into client values('C1003','Charlie','789 Ocean Drive','Tamilnadu',1000.00);
1 row created.
SQL> insert into client values('C1004','Daisy','321 River Road','Karnataka',3000.00);
1 row created.
SQL> insert into client values('C1005','Eva','654 lakeview blvd','Maharastra',2500.00);
1 row created.
SQL> insert into client values('C1006','Frank','888 Sky Ave','Kerala',1200.00);
1 row created.
SQL> insert into client values('C1007','Grace','999 Valley Rd','Karnataka',1800.00);
1 row created.
SQL> insert into client values('C1008','Henry','121 Sunset Blvd','Tamilnadu',2200.00);
1 row created.
SQL> insert into client values('C1009','Ivy','343 green St','Maharastra',1400.00);
1 row created.
SQL> insert into client values('C1010','Jack','676 Forest Lane','Kerala',1600.00);
1 row created.
```

```
SQL> select * from client;
```

CLIENT_NO	NAME	ADDRESS	STATE	BAL_DUE
C1001	Alice	123 Park lane	Karnataka	1500
C1002	Bob	456Hill street	Kerala	2000
C1003	Charlie	789 Ocean Drive	Tamilnadu	1000

C1004	Daisy	321 River Road	Karnataka	3000
C1005	Eva	654 lakeview blvd	Maharastra	2500
C1006	Frank	888 Sky Ave	Kerala	1200
C1007	Grace	999 Valley Rd	Karnataka	1800
C1008	Henry	121 Sunset Blvd	Tamilnadu	2200
C1009	Ivy	343 green St	Maharastra	1400
C1010	Jack	676 Forest Lane	Kerala	1600

10 rows selected.

a) From the table CLIENT, create a new table CLIENT1 that contains only CLIENT_NO and NAME, BAL_DUE from specified STATE. Accept the state during run time.

```
SQL> Accept state_input char prompt'enter state:'

enter state:karnataka

SQL> create table client1 as select client_no,name,bal_due from client where
state='&state_input';

old   1: create table client1 as select client_no,name,bal_due from client where
state='&state_input'

new   1: create table client1 as select client_no,name,bal_due from client where
state='karnataka'

Table created.
```

b) Create a new table CLIENT2 that has the same structure as CLIENT but with no records. Display the structure and records.

```
SQL> create table client2 as select * from client where 1=0;

Table created.

SQL> desc client2;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 CLIENT_NO                                          VARCHAR2(10)
 NAME                                      NOT NULL VARCHAR2(50)
```

ADDRESS		VARCHAR2(30)
STATE	NOT NULL	VARCHAR2(30)
BAL_DUE	NOT NULL	NUMBER(10,2)

SQL> select * from client2;

no rows selected

c) Add a new column by name PENALTY number (10, 2) to the CLIENT

SQL> alter table client add penalty number (10,2);

Table altered.

SQL> desc client;

Name	Null?	Type
CLIENT_NO	NOT NULL	VARCHAR2(10)
NAME	NOT NULL	VARCHAR2(50)
ADDRESS		VARCHAR2(30)
STATE	NOT NULL	VARCHAR2(30)
BAL_DUE	NOT NULL	NUMBER(10,2)
PENALTY		NUMBER(10,2)

d) Assign Penalty as 10% of BAL_DUE for the clients C1002, C1005, C1009 and for others 8%. Display Records

```
SQL> update client set penalty=case when client_no
in('C1002','C1005','C1009')then bal_due*0.10
  else bal_due*0.08
   end;
```

10 rows updated.

SQL> select * from client;

CLIENT_NO	NAME	ADDRESS	STATE	BAL_DUE	PENALTY
C1001	Alice	123 Park lane	Karnataka	1500	120
C1002	Bob	456 Hill street	Kerala	2000	200
C1003	Charlie	789 Ocean Drive	Tamilnadu	1000	80
C1004	Daisy	321 River Road	Karnataka	3000	240
C1005	Eva	654 lakeview	Maharastra	2500	250
C1006	Frank	888 Sky Ave	Kerala	1200	96
C1007	Grace	999 Valley Rd	Karnataka	1800	144
C1008	Henry	121 Sunset Blvd	Tamilnadu	2200	176
C1009	Ivy	343 green St	Maharastra	1400	140
C1010	Jack	676 Forest Lane	Kerala	1600	128

10 rows selected.

e) Change the name of CLIENT1 as NEW_CLIENT

SQL> alter table client1 rename to new_client;

Table altered.

f) Delete the table CLIENT2

SQL> drop table client2;

Table dropped.

Program 3:

Create a table BOOK using SQL command to store Accession No, TITLE, AUTHOR, PUBLISHER, YEAR, PRICE. Apply the suitable structure for the columns. Specify Primary Key and NOT NULL constraints on the table. Insert 10 records. Write the following SQL queries:

a) List the details of publishers having 'a' as the second character in their names.

b) Display Accession No., TITLE, PUBLISHER and YEAR of the books published by the specified author before 2010 in the descending order of YEAR. Accept author during run time

c) Modify the size of TITLE to increase the size 5 characters more.

d) Display the details of all books other than Microsoft press publishers.

e) Remove the records of the books published before 1990.

```
SQL> create table Book(
  2  AccessionNo varchar2(10) primary key,
  3  Title varchar2(50) not null,
  4  Author varchar2(50) not null,
  5  Publisher varchar2(50) not null,
  6  Year number(4) not null,
  7  Price number(8,2) not null);

Table created.

SQL> desc Book;
 Name                      Null?                Type
 ----------------------------------------- -------- ----------------------------
 ACCESSIONNO               NOT NULL             VARCHAR2(10)
 TITLE                     NOT NULL             VARCHAR2(50)
 AUTHOR                    NOT NULL             VARCHAR2(50)
```

PUBLISHER	NOT NULL	VARCHAR2(50)
YEAR	NOT NULL	NUMBER(4)
PRICE	NOT NULL	NUMBER(8,2)

SQL> insert into Book values('B001','Learning Sql','Alan Beaulieu','Oreilly',2009,450.00);

1 row created.

SQL> insert into Book values('B002','Sql in 10 minutes','Ben Forta','Sams',2012,550.00);

1 row created.

SQL> insert into Book values('B003','Head First Sql','lynn beighley','Oreilly',2007,600.00);

1 row created.

SQL> insert into Book values('B004','data base system','Thomas connolly','Pearson',2010,750.00);

1 row created.

SQL> insert into Book values('B005','Sql for dummies','Allen taylor','wiley',2008,400.00);

1 row created.

SQL> insert into Book values('B006','Pro Sql server','Adam machanic','Apress',2013,850.00);

1 row created.

SQL> insert into Book values('B007','T-sqlSql Fundamentals','Itzik Ben-Gan','Microsoft press',2009,670.00);

1 row created.

SQL> insert into Book values('B008','MySql CookBook','Paul Dubois','Oreilly',2011,580.00);

1 row created.

SQL> insert into Book values('B009','EffectiveSql','John Viescas','Addison-Wesley',2017,500.00);

1 row created.

SQL> insert into Book values('B010','Beginning Sql','Paul Wilton','Wrox',2003,430.00);

1 row created.

SQL> select * from Book;

ACCESSIONN	TITLE	AUTHOR	PUBLISHER	YEAR	PRICE
B001	Learning Sql	Alan Beaulieu	Oreilly	2009	450
B002	Sql in 10 minutes	Ben Forta	Sams	2012	550
B003	Head First Sql	lynn beighley	Oreilly	2007	600
B004	data base system	Thomas Connolly	Pearson	2010	750
B005	Sql for dummies	Allen taylor	wiley	2008	400
B006	Pro Sql server	Adam machanic	Apress	2013	850
B007	T-sqlSql Fundamentals	Itzik Ben-Gan	Microsoft press	2009	670
B008	MySql CookBook	Paul Dubois	Oreilly	2011	580
B009	EffectiveSql	John Viescas	Addison-Wesley	2017	500
B010	Beginning Sql	Paul Wilton	Wrox	2003	430

10 rows selected.

a) List the details of publishers having 'a' as the second character in their names.

SQL> select * from Book where publisher like'_a%';

ACCESSIONN	TITLE	AUTHOR	PUBLISHER	YEAR	PRICE
B002	Sql in 10 minutes	BenForta	Sams	2012	550

b) Display Accession No., TITLE, PUBLISHER and YEAR of the books published by the specified author before 2010 in the descending order of YEAR. Accept author during run time

SQL> select AccessionNo,Title,Publisher,Year from Book where Author='Alan Beaulieu' and year<2010 order by Year desc;

ACCESSIONN	TITLE	PUBLISHER	YEAR
B001	Learning Sql	Oreilly	2009

c) Modify the size of TITLE to increase the size 5 characters more.

SQL> alter table Book modify Title varchar2(55);

Table altered.

SQL> desc Book;

Name	Null?	Type
ACCESSIONNO	NOT NULL	VARCHAR2(10)
TITLE	NOT NULL	VARCHAR2(55)
AUTHOR	NOT NULL	VARCHAR2(50)
PUBLISHER	NOT NULL	VARCHAR2(50)
YEAR	NOT NULL	NUMBER(4)
PRICE	NOT NULL	NUMBER(8,2)

d) Display the details of all books other than Microsoft press publishers.

SQL> select * from book where publisher<>'Microsoft press';

ACCESSIONN	TITLE	AUTHOR	PUBLISHER	YEAR	PRICE
B001	Learning Sql	Alan Beaulieu	Oreilly	2009	450
B002	Sql in 10 minutes	Ben Forta	Sams	2012	550
B003	Head First Sql	lynn beighley	Oreilly	2007	600
B004	data base system	Thomas Connolly	Pearson	2010	750
B005	Sql for dummies	Allen taylor	wiley	2008	400
B006	Pro Sql server	Adam machanic	Apress	2013	850
B008	MySql CookBook	Paul Dubois	Oreilly	2011	580
B009	EffectiveSql	John Viescas	Addison-Wesley	2017	500
B010	Beginning Sql	Paul Wilton	Wrox	2003	430

09 rows selected.

e) Remove the records of the books published before 1990

SQL> delete from book where year<1990;

0 rows deleted.

Program 4:

Create a table SALES with columns SNO, SNAME, MNO, JOIN_DATE, DATE_BIRTH, SALARY, SALES_AMOUNT and COMMISSION. Minimum age for joining the company must be 18 Yrs. Default value for Commission should be 0. Apply the suitable structure for the columns. Specify Primary Key and NOT NULL constraints on the table. Insert 10 records with data except commission. Manager of Manager can be NULL. Write the following SQL queries:

a) Display the details of Sales Persons whose salary is more than Average salary in the company.

b) Update commission as 20% of Sales Amount.

c) Display SNO, SNAME, MNO, SALARY, COMMISSION, MANAGER_SALARY of the sales persons getting sum of salary and commission more than salary of manager. (Self-join)

d) Display the records of employees who finished the service of 10years

```
SQL> create table sales(
    sno number(4) primary key,
    sname varchar2(30) not null,
    manager_name varchar2(30),
    join_date date not null,
    date_birth date not null,
    salary number(10,2) not null,
    sales_amount number(10,2) not null,
    commission number(10,2) default 0 check(commission>=0),
   CONSTRAINT age_check CHECK(months_between(join_date,date_birth)>=216));
```

Table created.

SQL> desc sales;

Name	Null?	Type
SNO	NOT NULL	NUMBER(4)
SNAME	NOT NULL	VARCHAR2(30)
MANAGER_NAME		VARCHAR2(30)
JOIN_DATE	NOT NULL	DATE
DATE_BIRTH	NOT NULL	DATE
SALARY	NOT NULL	NUMBER(10,2)
SALES_AMOUNT	NOT NULL	NUMBER(10,2)
COMMISSION		NUMBER(10,2)

SQL> insert
intosales(sno,sname,manager_name,join_date,date_birth,salary,sales_amount)values

(1,'john','peter',TO_DATE('2015-06-15','YYYY-MM-DD'),TO_DATE('1990-05-20','YYYY-MM-DD'),50000,300000);

1 row created.

SQL> insert into
sales(sno,sname,manager_name,join_date,date_birth,salary,sales_amount)values

(2,'alice','peter',TO_DATE('2017-01-10','YYYY-MM-DD'),TO_DATE('1992-04-12','YYYY-MM-DD'),48000,250000);

1 row created.

SQL> insert into
sales(sno,sname,manager_name,join_date,date_birth,salary,sales_amount)values

(3,'david','john',TO_DATE('2016-09-25','YYYY-MM-DD'),TO_DATE('1991-11-01','YYYY-MM-DD'),47000,220000);

1 row created.

```
SQL> insert into
sales(sno,sname,manager_name,join_date,date_birth,salary,sales_amount)values

   (4,'sophia','alice',TO_DATE('2018-07-05','YYYY-MM-DD'),TO_DATE('1994-02-
18','YYYY-MM-DD'),46000,210000);

1 row created.


SQL> insert into
sales(sno,sname,manager_name,join_date,date_birth,salary,sales_amount)values

   (5,'liam','john',TO_DATE('2020-03-01','YYYY-MM-DD'),TO_DATE('1996-08-
25','YYYY-MM-DD'),45000,190000);

1 row created.


SQL> insert into
sales(sno,sname,manager_name,join_date,date_birth,salary,sales_amount)values

  (6,'Emma','peter',TO_DATE('2014-12-10','YYYY-MM-DD'),TO_DATE('1988-07-
14','YYYY-MM-DD'),55000,330000);

1 row created.


SQL> insert into
sales(sno,sname,manager_name,join_date,date_birth,salary,sales_amount)values

   (7,'noah','david',TO_DATE('2019-04-20','YYYY-MM-DD'),TO_DATE('1995-03-
02','YYYY-MM-DD'),44000,180000);

1 row created.


SQL> insert into
sales(sno,sname,manager_name,join_date,date_birth,salary,sales_amount)values

  (8,'olivia','sophia',TO_DATE('2021-06-30','YYYY-MM-DD'),TO_DATE('2000-01-
01','YYYY-MM-DD'),43000,175000);

1 row created


SQL> insert into
sales(sno,sname,manager_name,join_date,date_birth,salary,sales_amount)values
```

(9,'william','liam',TO_DATE('2013-11-15','YYYY-MM-DD'),TO_DATE('1985-10-10','YYYY-MM-DD'),52000,295000);

1 row created.

SQL> insert into sales(sno,sname,manager_name,join_date,date_birth,salary,sales_amount)values

(10,'ava','null',TO_DATE('2012-08-01','YYYY-MM-DD'),TO_DATE('1980-05-05','YYYY-MM-DD'),60000,350000);

1 row created.

SQL> select * from sales;

SNO	SNAME	MANAGER_NAME	JOIN_DATE	DATE_BIRT	SALARY	SALES_AMOUNT	COMMISSION
1	john	peter	15-JUN-15	20-MAY-90	50000	300000	0
2	alice	peter	10-JAN-17	12-APR-92	48000	250000	0
3	david	john	25-SEP-16	01-NOV-91	47000	220000	0
4	sophia	alice	05-JUL-18	18-FEB-94	46000	210000	0
5	liam	john	01-MAR-20	25-AUG-96	45000	190000	0
6	Emma	peter	10-DEC-14	14-JUL-88	55000	330000	0
7	noah	David	20-APR-19	02-MAR-95	44000	180000	0
8	olivia	Sophia	30-JUN-21	01-JAN-00	43000	175000	0
9	william	liam	15-NOV-13	10-OCT-85	52000	295000	0
10	ava	null	01-AUG-12	05-MAY-80	60000	350000	0

10 rows selected.

a) Display the details of Sales Persons whose salary is more than Average salary in the company.

SQL> select * from sales where salary>(select avg(salary)from sales);

SNO	SNAME	MANAGER_NAME	JOIN_DATE	DATE_BIRT	SALARY	SALES_AMOUNT	COMMISSION
1	john	peter	15-JUN-15	20-MAY-90	50000	300000	0
6	Emma	peter	10-DEC-14	14-JUL-88	55000	330000	0
9	william	liam	15-NOV-13	10-OCT-85	52000	295000	0
10	ava	null	01-AUG-12	05-MAY-80	60000	350000	0

b) Update commission as 20% of Sales Amount.

SQL> update sales set commission =sales_amount*0.20;

10 rows updated.

SQL> select * from sales;

SNO	SNAME	MANAGER_NAME	JOIN_DATE	DATE_BIRT	SALARY	SALES_AMOUNT	COMMISSION
1	john	peter	15-JUN-15	20-MAY-90	50000	300000	60000
2	alice	peter	10-JAN-17	12-APR-92	48000	250000	50000
3	david	john	25-SEP-16	01-NOV-91	47000	220000	44000
4	sophia	alice	05-JUL-18	18-FEB-94	46000	210000	42000
5	liam	john	01-MAR-20	25-AUG-96	45000	190000	38000
6	Emma	peter	10-DEC-14	14-JUL-88	55000	330000	66000
7	noah	David	20-APR-19	02-MAR-95	44000	180000	36000
8	olivia	Sophia	30-JUN-21	01-JAN-00	43000	175000	35000
9	william	liam	15-NOV-13	10-OCT-85	52000	295000	59000
10	ava	null	01-AUG-12	05-MAY-80	60000	350000	70000

10 rows selected.

d) Display the records of employees who finished the service of 10years

SQL> select * from sales where add_months(join_date,120)<=sysdate;

SNO	SNAME	MANAGER_NAME	JOIN_DATE	DATE_BIRT	SALARY	SALES_AMOUNT	COMMISSION
1	john	peter	15-JUN-15	20-MAY-90	50000	300000	60000
6	Emma	peter	10-DEC-14	14-JUL-88	55000	330000	66000
9	william	liam	15-NOV-13	10-OCT-85	52000	295000	59000
10	ava	null	01-AUG-12	05-MAY-80	60000	350000	70000

c) Display SNO, SNAME, MNO, SALARY, COMMISSION, MANAGER_SALARY of the sales persons getting sum of salary and commission more than salary of manager. (Self-join)

SQL> select e.sno,e.sname,e.manager_name,e.salary,e.commission,m.salary as manager_salary from sales e join sales m on e.manager_name=m.sname where(e.salary+e.commission)>m.salary;

SNO	SNAME	MANAGER_NAME	SALARY	COMMISSION	MANAGER_SALARY
5	liam	john	45000	38000	50000
3	david	john	47000	44000	50000
4	sophia	alice	46000	42000	48000
7	noah	david	44000	36000	47000
8	olivia	Sophia	43000	35000	46000
9	william	liam	52000	59000	45000

6 rows selected.

Program 5:

Create a table Sales_Details with the columns SNO, MONTH, TARGET and QTY_SOLD to store the Sales Details of one year. Specify the composite primary key to the columns SNO and MONTH. TARGET and SALES must be positive numbers. Write the following SQL queries:

a) Display the total sales by each sales person considering only those months sales where target was reached

b) If a commission of RS.50 provided for each item after reaching target, calculate and display the total commission for each sales person.

c) Display the SNO of those who never reached the target.

d) Display the SNO, MONTH and QTY_SOLD of the sales persons with SNO S0001 or S0003

```
SQL> create table Sales_Details(
  2  sno varchar(10),
  3  month varchar(10),
  4  target number(10,2)check(target>0),
  5  qty_sold number(10)check(qty_sold>0),
  6  primary key(sno,month));
Table created.

SQL> desc Sales_Details;
 Name                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 SNO                       NOT NULL VARCHAR2(10)
 MONTH                     NOT NULL VARCHAR2(10)
 TARGET                             NUMBER(10,2)
 QTY_SOLD                           NUMBER(10)
```

```
SQL> insert into Sales_Details values('S0001','Jan',100,120);

1 row created.

SQL> insert into Sales_Details values('S0001','Feb',150,130);

1 row created.

SQL> insert into Sales_Details values('S0002','Jan',100,90);

1 row created.

SQL> insert into Sales_Details values('S0002','Feb',150,160);

1 row created.

SQL> insert into Sales_Details values('S0003','Jan',120,130);

1 row created.

SQL> insert into Sales_Details values('S0003','Feb',100,80);

1 row created.


SQL> select * from Sales_Details;

SNO        MONTH          TARGET   QTY_SOLD
S0001          Jan              100       120
S0001          Feb              150       130
S0002          Jan              100        90
S0002          Feb              150       160
S0003          Jan              120       130
S0003          Feb              100        80

6 rows selected.
```

a) Display the total sales by each sales person considering only those months sales where target was reached

```
SQL> select sno,sum(qty_sold)as total_sales from Sales_Details where qty_sold>=target
group by sno;

SNO        TOTAL_SALES
---------- -----------
S0001            120
```

S0002 160

S0003 130

b) If a commission of RS.50 provided for each item after reaching target, calculate and display the total commission for each sales person.

SQL> select sno,sum((qty_sold-target) *50) as total_commission from Sales_Details where qty_sold>target group by sno;

SNO TOTAL_COMMISSION

---------- ----------

S0001 1000

S0002 500

S0003 500

c) Display the SNO of those who never reached the target.

SQL> select sno from Sales_Details group by sno having max(qty_sold) <min(target);

no rows selected

d) Display the SNO, MONTH and QTY_SOLD of the sales persons with SNO S0001 or S0003.

SQL> select sno,month,qty_sold from Sales_Details where sno in('S0001','S0003');

SNO MONTH QTY_SOLD

---------- ---------- ----------

S0001 Feb 130

S0001 Jan 120

S0003 Feb 80

S0003 Jan 130

Program 6:

Create the following tables by identifying primary and foreign keys. Specify the not null property for mandatory keys. SUPPLIERS (SUPPLIER_NO, SNAME, SADDRESS, SCITY) COMPUTER_ITEMS (ITEM_NO, SUPPLIER_NO, ITEM_NAME, IQUANTITY) Consider three suppliers. A supplier can supply more than one type of items. Write the SQL queries for the following:

a) List ITEM and SUPPLIER details in alphabetical order of city name and in each city decreasing order of IQUANTITY.

b) List the name, city,and address of the suppliers who are supplying keyboard.

c) List the supplier's name, items supplied by the suppliers 'Cats' and 'Electrotech'.

d) Find the items having quantity less than 5 and insert the details of supplier and item of these, into another table NEWORDER

```
SQL> create table suplier(
  2  supplier_no varchar(10) primary key,
  3  sname varchar(50) not null,
  4  saddress varchar(100),
  5  scity varchar(50) not null);
Table created.


SQL> desc suplier;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 SUPPLIER_NO                               NOT NULL VARCHAR2(10)
 SNAME                                     NOT NULL VARCHAR2(50)
 SADDRESS                                           VARCHAR2(100)
 SCITY                                     NOT NULL VARCHAR2(50)
```

```
SQL> create table computer_items(
  2  item_no varchar(10) primary key,
  3  supplier_no varchar(10) not null,
  4  item_name varchar(50) not null,
  5  Iquantity number(5)check(Iquantity>=0),
  6  Foreign key(supplier_no)references suplier(supplier_no));
Table created.

SQL> desc computer_items;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ITEM_NO                                   NOT NULL VARCHAR2(10)
 SUPPLIER_NO                               NOT NULL VARCHAR2(10)
 ITEM_NAME                                 NOT NULL VARCHAR2(50)
 IQUANTITY                                          NUMBER(5)

SQL> insert into suplier values('S1','cats','12 main st','Delhi');
1 row created.
SQL> insert into suplier values('S2','electrotech','45 tech ave','Mumbai');
1 row created.
SQL> insert into suplier values('S3','digitech','78 supply rd','Delhi');
1 row created.
SQL> select * from suplier;
SUPPLIER_N SNAME      SADDRESS       SCITY
S1         cats       12 main st     Delhi
S2         electrotech 45 tech ave   Mumbai
S3         digitech   78 supply rd   Delhi
```

```
SQL> insert into computer_items values('I1','S1','Mouse',10);

1 row created.

SQL> insert into computer_items values('I2','S1','Keyboard',3);

1 row created.

SQL> insert into computer_items values('I3','S2','Monitor',15);

1 row created.

SQL> insert into computer_items values('I4','S2','Keyboard',2);

1 row created.

SQL> insert into computer_items values('I5','S3','CPU',8);

1 row created.

SQL> select * from computer_items;
```

ITEM_NO	SUPPLIER_N	ITEM_NAME	IQUANTITY
I1	S1	Mouse	10
I2	S1	Keyboard	3
I3	S2	Monitor	15
I4	S2	Keyboard	2
I5	S3	CPU	8

a) List ITEM and SUPPLIER details in alphabetical order of city name and in each city decreasing order of IQUANTITY.

```
SQL> select ci.item_name,ci.Iquantity,s.supplier_no,s.sname,s.saddress,s.scity from
computer_items ci join suplier s on ci.supplier_no=s.supplier_no order by s.scity
asc,ci.Iquantity desc;
```

ITEM_NAME	IQUANTITY	SUPPLIER_N	SNAME	SADDRESS	SCITY
Mouse	10	S1	cats	12 main st	Delhi
CPU	8	S3	digitech	78 supply rd	Delhi
Keyboard	3	S1	cats	12 main st	Delhi
Monitor	15	S2	electrotech	45 tech ave	Mumbai
Keyboard	2	S2	electrotech	45 tech ave	Mumbai

b) List the name ,city, and address of the suppliers who are supplying keyboard.

SQL> select distinct s.sname,s.scity,s.saddress from suplier s join computer_items ci on s.supplier_no=ci.supplier_no where lower(ci.item_name)='keyboard';

SNAME	SCITY	SADDRESS
Cats	Delhi	12 main st
Electrotech	Mumbai	45 tech ave

c) List the supplier's name, items supplied by the suppliers 'Cats' and 'Electrotech'.

SQL> select s.sname,ci.item_name from suplier s join computer_items ci on s.supplier_no=ci.supplier_no where s.sname in('cats','electrotech');

SNAME	ITEM_NAME
Cats	Mouse
Cats	Keyboard
Electrotech	Monitor
Electrotech	Keyboard

d) Find the items having quantity less than 5 and insert the details of supplier and item of these, into another table NEWORDER

SQL> create table neworder(

2 supplier_no varchar(10),

3 item_no varchar(10),

4 item_name varchar(50),

5 Iquantity number(5),

6 Foreign key(supplier_no)references suplier(supplier_no));

Table created.

SQL> insert into neworder(supplier_no,item_no,item_name,Iquantity) select ci.supplier_no,ci.item_no,ci.item_name,ci.Iquantity from computer_items ci where ci.Iquantity<5;

2 rows created.

SQL> select * from neworder;

SUPPLIER_N	ITEM_NO	ITEM_NAME	IQUANTITY
S1	I2	Keyboard	3
S2	I4	Keyboard	2

Program 7:

Create the following tables by identifying primary and foreign keys, specify the not null property for mandatory keys.

PRODUCT_DETAIL				
P_NO	**PRODUCTNAME**	**QTYAVAILABLE**	**PRICE**	**PROFIT %**
P0001	Monitor	10	3000	20
P0002	Pen Drives	50	650	5
P0003	CD Drive	100	10	3
P0004	Key Board	25	600	10

PURCHASED_DETAIL		
CUSTNO	**P_NO**	**QTYSOLD**
C1	P0003	2
C2	P0002	4
C3	P0002	10
C4	P0001	3
C1	P0004	2
C2	P0003	2
C4	P0004	1

Write the following SQL queries:

a) Display total amount spent by C2.

b) Display the names of product for which either QtyAvailable is less than 30 or total QtySold is less than 5(USE UNION).

c) Display the name of products and quantity purchased by C4.

d) How much Profit does the shopkeeper gets on C1's purchase?

e) How many 'Pen Drives' have been sold?

```
SQL> create table product_detaill(
  2  p_no varchar(10)primary key,
  3  productname varchar(50)not null,
  4  qtyavailable number not null,
  5  price number not null,
  6  profit_percent number);
```

Table created.

SQL> desc product_detaill;

Name	Null?	Type
P_NO	NOT NULL	VARCHAR2(10)
PRODUCTNAME	NOT NULL	VARCHAR2(50)
QTYAVAILABLE	NOT NULL	NUMBER
PRICE	NOT NULL	NUMBER
PROFIT_PERCENT		NUMBER

SQL> create table purchased_detail(

2 custno varchar(10) not null,

3 p_no varchar(10) not null,

4 qtysold number not null,

5 foreign key(p_no)references product_detaill(p_no));

Table created.

SQL> desc purchased_detail;

Name	Null?	Type
CUSTNO	NOT NULL	VARCHAR2(10)
P_NO	NOT NULL	VARCHAR2(10)
QTYSOLD	NOT NULL	NUMBER

SQL> insert into product_detaill values('p0001','monitor',10,30000,20);

1 row created.

SQL> insert into product_detaill values('p0002','pendrives',50,650,5);

1 row created.

SQL> insert into product_detaill values('p0003','CD drive',100,10,3);

1 row created.

SQL> insert into product_detaill values('p0004','keyboard',25,600,10);

1 row created.

SQL> select * from product_detaill;

P_NO	PRODUCTNAME	QTYAVAILABLE	PRICE	PROFIT_PERCENT
p0001	monitor	10	30000	20
p0002	pendrives	50	650	5
p0003	CD drive	100	10	3
p0004	keyboard	25	600	10

SQL> insert into purchased_detail values('c1','p0003',2);

1 row created.

SQL> insert into purchased_detail values('c2','p0002',4);

1 row created.

SQL> insert into purchased_detail values('c3','p0002',10);

1 row created.

SQL> insert into purchased_detail values('c4','p0001',3);

1 row created.

SQL> insert into purchased_detail values('c1','p0004',2);

1 row created.

SQL> insert into purchased_detail values('c2','p0003',2);

1 row created.

SQL> insert into purchased_detail values('c4','p0004',1);

1 row created.

SQL> select * from purchased_detail;

CUSTNO	P_NO	QTYSOLD
c1	p0003	2
c2	p0002	4
c3	p0002	10
c4	p0001	3
c1	p0004	2
c2	p0003	2
c4	p0004	1

7 rows selected.

a) Display total amount spent by C2.

```
SQL> select custno,sum(qtysold*price) as total_amount_spent
    from purchased_detail pd
    join product_detaill p on pd.p_no=p.p_no
    where custno='c2'
   group by custno;
```

CUSTNO	TOTAL_AMOUNT_SPENT
c2	2620

b) Display the names of product for which either QtyAvailable is less than 30 or total QtySold is less than 5(USE UNION).

```
SQL> select productname from product_detaill where qtyavailable<30
    union
    select p.productname
    from product_detaill p
    join purchased_detail pd on p.p_no=pd.p_no
    group by p.productname
    having sum(qtysold)<5;
```

PRODUCTNAME

CD drive

keyboard

monitor

c) Display the name of products and quantity purchased by C4.

SQL> select p.productname,pd.qtysold from purchased_detail pd join product_detaill p on pd.p_no=p.p_no where pd.custno='c4';

PRODUCTNAME	QTYSOLD
monitor	3
keyboard	1

d) How much Profit does the shopkeeper gets on C1's purchase?

SQL> select sum((p.price*pd.qtysold)*(p.profit_percent/100)) as total_profit from purchased_detail pd join product_detaill p on pd.p_no=p.p_no where pd.custno='c1';

TOTAL_PROFIT

120.6

e) How many 'Pen Drives' have been sold?

SQL> select sum(pd.qtysold)as total_pen_drives_sold from purchased_detail pd join product_detaill p on pd.p_no=p.p_no where p.productname='pendrives';

TOTAL_PEN_DRIVES_SOLD

14

PART-B

Program 1:

Create a table Bank with the columns ACNO, ACT_NAME, ACT_TYPE and BAL. Specify the Primary Key. Initial BAL must be greater than 500. Write a PL/SQL program to perform debit operation by providing acct_no and amount required. The amount must be greater than 100 and less than 20000 for one transaction. If the account exists and BAL-amount>500 Bank table must be updated, otherwise "NO SUFFFICIENT BALANCE" message should be displayed. If account number is not present then display "NO SUCH ACCOUNT" message to the user.

```
SQL> create table bank(
  2  accno number(15)primary key,
  3  act_name varchar2(30)not null,
  4  act_type varchar2(30)not null,
  5  bal number(10,2)check(bal>500));

Table created.

SQL> desc bank;
 Name                      Null?                Type
 ----------------------------------------- -------- ----------------------------
 ACCNO                     NOT NULL             NUMBER(15)
 ACT_NAME                  NOT NULL             VARCHAR2(30)
 ACT_TYPE                  NOT NULL             VARCHAR2(30)
 BAL                                            NUMBER(10,2)

SQL> insert into bank values(10001,'pruthvi','savings',7000);
1 row created.
SQL> insert into bank values(10002,'anup','IT',70000);
1 row created.
```

```
SQL> insert into bank values(10003,'yunith','savings',60000);
1 row created.
SQL> insert into bank values(10004,'bhandavi','manager',50000);
1 row created.
SQL> insert into bank values(10005,'hardik','manager',70000);
1 row created.
SQL> insert into bank values(10006,'sameeksha','IT',90000);
1 row created.
SQL> insert into bank values(10007,'shreyas','savings',20000);
1 row created.

SQL> select * from bank;
```

ACCNO	ACT_NAME	ACT_TYPE	BAL
10001	pruthvi	savings	7000
10002	anup	IT	70000
10003	yunith	savings	60000
10004	bhandavi	manager	50000
10005	hardik	manager	70000
10006	sameeksha	IT	90000
10007	shreyas	savings	20000

7 rows selected.

SQL> ed Bank.sql

```
set serveroutput ON;
ACCEPT acno NUMBER PROMPT "Enter the Account Number:"
ACCEPT amount1 NUMBER PROMPT "Enter the Amount to Withdraw:"
DECLARE
        acno number(10);
        amount1 number(10);
```

```
        db_accno number(10);
        db_amount number(10);
        db_name varchar(25);
BEGIN
        acno:=&acno;
        amount1:=&amount1;
  SELECT  accno,bal INTO  db_accno,db_amount
  FROM bank
  WHERE accno = acno;
IF (amount1>100 AND amount1<20000) then
        IF db_amount>100 then
                UPDATE bank
                SET bal=bal-amount1
                WHERE accno=acno;
                COMMIT;
                Dbms_output.put_line('WITHDRAWAL SUCCESS.');
                SELECT  accno,bal,act_name INTO  db_accno,db_amount,db_name
                FROM bank
                WHERE accno = acno;
                DBMS_OUTPUT.PUT_LINE ('Available Balance: '|| db_amount);
                DBMS_OUTPUT.PUT_LINE ('Name: '|| db_name);
                DBMS_OUTPUT.PUT_LINE ('Account Number: '|| db_accno);
        ELSE
                Dbms_output.put_line('NO SUFFICIENT BALANCE!');
        END IF;
ELSE
        Dbms_output.put_line('INVALID AMOUNT');
END IF;
EXCEPTION
```

```
   WHEN no_data_found THEN
      dbms_output.put_line('NO SUCH ACCOUNT!');
END;
/
```

OUTPUT:

```
set serveroutput ON;
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\bank.sql";
Enter the Account Number:101
Enter the Amount to Withdraw:2000
old   8:       acno:=&acno;
new   8:        acno:=       101;
old   9:       amount1:=&amount1;
new   9:        amount1:=      2000;
```

NO SUCH ACCOUNT!

```
PL/SQL procedure successfully completed.

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\bank.sql";
Enter the Account Number:10001
Enter the Amount to Withdraw:200
old   8:       acno:=&acno;
new   8:        acno:=     10001;
old   9:       amount1:=&amount1;
new   9:        amount1:=       200;
```

WITHDRAWAL SUCCESS.

```
Available Balance: 6800
Name: pruthvi
Account Number: 10001
PL/SQL procedure successfully completed.
```

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\bank.sql";

Enter the Account Number:10006

Enter the Amount to Withdraw:900000

old 8: acno:=&acno;

new 8: acno:= 10006;

old 9: amount1:=&amount1;

new 9: amount1:= 900000;

INVALID AMOUNT

PL/SQL procedure successfully completed.

Program 2:

Create a table STOCK_DETAIL with the columns PNO, PNAME and QTY_AVL to store stock details of computer accessories. Specify Primary Key and NOT NULL constraints on the table. QTY_AVL should be positive number. Write a PL/SQL Program to define a user defined exception named "LOW_STOCK" to validate the transaction. The program facilitates the user to purchase the product by providing product number and quantity required. It should display an error message "NO SUFFICIENT STOCK" when the user tries to purchase a product with quantity more than QTY_AVL, Otherwise the STOCK_DETAIL table should be updated for valid transaction.

```
SQL> create table stock_detail(
  2  pno varchar2(10) primary key,
  3  pname varchar2(20) not null,
  4  qty_avl number(30)check(qty_avl>0));

Table created.

SQL> desc stock_detail;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 PNO                                       NOT NULL VARCHAR2(10)
 PNAME                                     NOT NULL VARCHAR2(20)
 QTY_AVL                                            NUMBER(30)

SQL> insert into stock_detail values('101','computer',50);
1 row created.
SQL> insert into stock_detail values('102','keyboard',30);
1 row created.
SQL> insert into stock_detail values('103','mouse',50);
1 row created.
```

```
SQL> insert into stock_detail values('104','cpu',60);
1 row created.
SQL> insert into stock_detail values('105','pendrive',80);
1 row created.
```

SQL> ed stock_detail.sql

```
set serveroutput ON;
ACCEPT prdtNo CHAR PROMPT "Enter the Product Number:"
ACCEPT Qty NUMBER PROMPT "Enter the Quantity Required:"
DECLARE
 prdtNo varchar2(10);
prodName varchar2(25);
Qty number(10);
db_prodNo varchar2(10);
db_prodQty number(10);
LOW_STOCK EXCEPTION;
BEGIN
prdtNo:='&prdtNo';
Qty:=&Qty;
 SELECT pno,pname,qty_avl INTO db_prodNo,prodName,db_prodQty
 FROM stock_detail
 WHERE pno = prdtNo;
DBMS_OUTPUT.PUT_LINE ('Available Stock: '|| db_prodQty);
DBMS_OUTPUT.PUT_LINE ('Product Name: '|| prodName);
IF (Qty<db_prodQty) then
UPDATE stock_detail
SET qty_avl=qty_avl-Qty
WHERE pno=prdtNo;
COMMIT;
```

```
Dbms_output.put_line('Values Updated');
 SELECT pno,pname,qty_avl INTO db_prodNo,prodName,db_prodQty
 FROM stock_detail
 WHERE pno = prdtNo;
 DBMS_OUTPUT.PUT_LINE ('Available Stock: '|| db_prodQty);
 DBMS_OUTPUT.PUT_LINE ('Product Name: '|| prodName);
ELSE
raise LOW_STOCK;
END IF;
EXCEPTION
 WHEN no_data_found THEN
 dbms_output.put_line('NO SUCH PRODUCT!');
 WHEN LOW_STOCK THEN
dbms_output.put_line('NO SUFFICIENT STOCK!');
END;
/
```

OUTPUT:

```
SQL> set serveroutput ON;
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\stock_detail.sql";
Enter the Product Number:101
Enter the Quantity Required:20
old   9: prdtNo:='&prdtNo';
new   9: prdtNo:='101';
old  10: Qty:=&Qty;
new  10: Qty:=        20;
Available Stock: 50
Product Name: computer
Values Updated
Available Stock: 30
```

Product Name: computer

PL/SQL procedure successfully completed.

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\stock_detail.sql";

Enter the Product Number:p101

Enter the Quantity Required:30

old 9: prdtNo:='&prdtNo';

new 9: prdtNo:='p101';

old 10: Qty:=&Qty;

new 10: Qty:= 30;

NO SUCH PRODUCT!

PL/SQL procedure successfully completed.

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\stock_detail.sql";

Enter the Product Number:103

Enter the Quantity Required:90

old 9: prdtNo:='&prdtNo';

new 9: prdtNo:='103';

old 10: Qty:=&Qty;

new 10: Qty:= 90;

Available Stock: 60

Product Name: cpu

NO SUFFICIENT STOCK!

PL/SQL procedure successfully completed.

Program 3:

Write a PL/SQL program to compute the selling price of books depending on the book code and category. Use Open, Fetch and Close. The Book_detail table contains columns: Book Code, Author, Title, Category and Price. Insert 10 records. The selling price=Price-Discount. The discount is calculated as follows:

The discount is calculated as follows:

Book Code	Category	Discount Percentage
A	Novels	10% of Price
	Technology	12.5% of Price
B	Commerce	18% of Price
	Science	19% of Price
C	Songs	25% of Price
	Sports	24% of Price
D	All	28% of Price

Print the result in tabular form with proper alignment

```
Book Code  category  title  author  price  discount %  discount amount  sell price
=========  ========  ======  =====  ======  =======  ======
-------------------------------------------------------------------------------------
```

```
SQL> create table book_details(
    bkcode varchar2(2),
    author varchar2(20)not null,
    title varchar2(20)not null,
    category varchar2(20)not null,
    price number(10)not null);

Table created.

SQL> desc book_details;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 BKCODE                                             VARCHAR2(2)
 AUTHOR                                    NOT NULL VARCHAR2(20)
```

TITLE	NOT NULL	VARCHAR2(20)
CATEGORY	NOT NULL	VARCHAR2(20)
PRICE	NOT NULL	NUMBER(10)

```
SQL> insert into book_details values('a','balaguruswamy','cprogam','technology',220);
1 row created.
SQL> insert into book_details values('a','swami','computernetwork','technology',230);
1 row created.
SQL> insert into book_details values('b','sharada','accounts','commerce',1200);
1 row created.
SQL> insert into book_details values('b','johnwilson','science','mathematics',1200);
1 row created.
SQL> insert into book_details values('c','balasubramanyam','devotional','songs',300);
1 row created.
SQL> insert into book_details values('c','viratkohli','sports spirit','sports',300);
1 row created.
SQL> insert into book_details values('d','viratkohli','sports spirit','sports',300);
1 row created.
SQL> insert into book_details values('d','shilpa','theteaderate','economics',700);
1 row created.
SQL> insert into book_details values('d','kruthi','businessworld','business',800);
1 row created.

SQL> select * from book_details;
```

BK	AUTHOR	TITLE	CATEGORY	PRICE
a	balaguruswamy	cprogam	technology	220
a	swami	computernetwork	technology	230
b	sharada	accounts	commerce	1200

b	johnwilson	science	mathematics	1200
c	balasubramanyam	devotional	songs	300
c	viratkohli	sports spirit	sports	300
d	viratkohli	sports spirit	sports	300
d	shilpa	theteaderate	economics	700
d	kruthi	businessworld	business	800

9 rows selected.

SQL> ed book1.sql

```
Set serveroutput ON;
DECLARE
bkCode varchar2(10);
bkAuthor varchar2(20);
bkTitle varchar2(20);
bkCategory varchar2(20);
bkDiscount number(10,1);
bkPrice number(10,1);
bkDisAmount number(10,1);
bkSell number(10,2);
CURSOR bk_details IS SELECT bkcode, author, title,category,price FROM book_details;
BEGIN
 OPEN bk_details;
dbms_output.put_line(LPAD('-',150,'-'));dbms_output.put_line(RPAD('Book Code',20)||
RPAD('Category',20)||RPAD('Title',20)||RPAD('Author',20)||RPAD('Price',20)||RPAD('discou
nt %',20)||RPAD('discount amount',20)||RPAD('sell price',20));
dbms_output.put_line(LPAD('-',150,'-'));
LOOP
 FETCH bk_details into bkCode,bkAuthor,bkTitle,bkCategory,bkPrice;
EXIT WHEN bk_details%notfound;
IF(bkCode='A' AND bkCategory='Novels') then
```

```
bkDiscount:=10;
ELSIF(bkCode='A' AND bkCategory='Technology') then
bkDiscount:=12.5;
ELSIF(bkCode='B' AND bkCategory='Commerce') then
bkDiscount:=18;
ELSIF(bkCode='B' AND bkCategory='Science') then
bkDiscount:=19;
ELSIF(bkCode='C' AND bkCategory='Songs') then
bkDiscount:=25;
ELSIF(bkCode='C' AND bkCategory='Sports') then
bkDiscount:=24;
ELSE
bkDiscount:=28;
END IF;
bkDisAmount:=bkPrice*bkDiscount/100;
bkSell:=bkPrice-bkDisAmount;
dbms_output.put_line(RPAD(bkCode,20)||RPAD(bkCategory,20)||RPAD(bkTitle,20)||RPAD(
bkAuthor,20)||RPAD(bkPrice,20)||RPAD(bkDiscount,20)||RPAD(bkDisAmount,20)||RPAD(b
kSell,20));
 END LOOP;
dbms_output.put_line(LPAD('-',150,'-'));
CLOSE bk_details;
END;
/
```

OUTPUT:

```
SQL>set serveroutput ON;
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\book1.sql";
```

Book Code	Category	Title	Author	Price	discount %	discountAmount	sellprice
a	technology	cprogam	balaguruswamy	220	28	61.6	158.4
a	technology	computernetwork	swami	230	28	64.4	165.6
b	commerce	accounts	sharada	1200	28	336	864
b	mathematics	science	johnwilson	1200	28	336	864
c	songs	devotional	balasubramanyam	300	28	84	216
c	sports	sports spirit	viratkohli	300	28	84	216
d	sports	sports spirit	viratkohli	300	28	84	216
d	economics	theteaderate	Shilpa	700	28	196	504
d	business	businessworld	kruthi	800	28	224	576

```
PL/SQL procedure successfully completed.
```

Program 4:

Write a PL/SQL program to display employee pay bill (using Cursor For loop) Use a Procedure to receive basic pay and to compute DA, HRA, Tax, PF, Gross Pay and Net Pay(Use OUT). Base table contains the following columns empnum, empname, basic pay. Insert 3 records. Allowances are computed as follows

Basic Pay	DA	HRA
<=20000	35% of Basic	8% of Basic
>20000 & <=30000	38%	9%
>30000 & <=40000	40%	10%
>40000	45%	10%

Gross=Basic+DA+HRA
PF=12% of Gross or Rs. 2000 whichever is minimum.
PT=Rs. 100 upto Gross is 25,000 else Rs. 200.
Net=Gross-(PF+PT)
Print Pay slip as follows.

```
----------PAYSLIP----------
Empno          :10011          Empname : Raj
Basic Pay      :20000            P.F.  : 3432
DA             :7000             P.T.  : 200
H.R.A.         :1600
Gross          :28600        Net Pay  : 24968
--------------------------------------------
----------PAYSLIP----------
Empno          :10012          Empname : Rani
Basic Pay      :30000            P.F.  : 5292
DA             :11400            P.T.  : 200
H.R.A.         :2700
Gross          :44100        Net Pay  : 38608
--------------------------------------------
```

SQL> connect system/tiger

Connected.

SQL> create table employee(

2 empnum varchar2(20)primary key,

3 empname varchar2(30)not null,

4 basicpay number(10,2)not null);

Table created.

SQL> desc employee;

Name	Null?	Type
EMPNUM	NOT NULL	VARCHAR2(20)
EMPNAME	NOT NULL	VARCHAR2(30)

BASICPAY NOT NULL NUMBER(10,2)

```
SQL> insert into employee values('e101','johnsmith',26000.78);
1 row created.
SQL> insert into employee values('e102','jerold wilson',34500.67);
1 row created.
SQL> insert into employee values('e103','ramachandra',44580.56);
1 row created.
SQL> insert into employee values('e104','anas',15580.39);
1 row created.

SQL> select * from employee;
EMPNUM          EMPNAME          BASICPAY
-------------------- ------------------------------ ----------
e101            johnsmith                26000.78
e102            jerold wilson            34500.67
e103            ramachandra              44580.56
e104            anas                     15580.39
```

SQL> ed employee.sql

```
set serveroutput on;
declare
        cursor emp_cur is select empnum,empname,basicpay from employee;
        eno varchar2(20);
        name varchar2(30);
        bs number(10,2);
        da number(10,2);
        hra number(10,2);
        pt number(10,2);
```

```
        pf number(10,2);
        gross number(10,2);
        np number(10,2);
begin
        open emp_cur;
loop
        fetch emp_cur into eno,name,bs;
        exit when emp_cur%notfound;
        if bs<=20000 then
                da:=bs*0.35;
                hra:=bs*0.8;
        elsif(bs>20000 and bs<=30000)then
                da:=bs*0.38;
                hra:=bs*0.9;
        elsif(bs>30000 and bs<=40000)then
                da:=bs*0.40;
                hra:=bs*0.10;
        elsif(bs>40000)then
                da:=bs*0.45;
                hra:=bs*0.10;
        end if;
        gross:=bs+da+hra;
        pf:=gross*0.12;
        if pf>=2000 then
                pf:=2000;
        end if;
        if gross<=25000 then
                pt:=100;
        else
```

```
                pt:=200;
        end if;
        np:=gross-(pf+pt);
        dbms_output.put_line( LPAD('=',30,'=')||'PAY SLIP'||LPAD('=',30,'='));
        dbms_output.put_line(RPAD('Emp no' ,10)||RPAD(eno,10)||RPAD('Emp name
:',10)||RPAD(name,10));
        dbms_output.put_line(RPAD('Basicpay' ,10)||RPAD(bs,10)||RPAD('PF
:',10)||RPAD(pf,10));
        dbms_output.put_line(RPAD('DA' ,10)||RPAD(da,10)||RPAD('PT
:',10)||RPAD(pt,10));
        dbms_output.put_line(RPAD('HRA' ,10)||RPAD(hra,10));
        dbms_output.put_line(RPAD('Gross' ,10)||RPAD(gross,10)||RPAD('Net
Pay',10)||RPAD(np,10));
        dbms_output.put_line( LPAD('=',70,'='));
end loop;
close emp_cur;
end;
/
```

OUTPUT:

```
SQL> set serveroutput ON;
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\employee.sql";
==============================PAYSLIP==============================
Empno    e101          Emp name :johnsmith
Basicpay  26000.78     PF :      2000
DA :       9880.3      PT :      200
HRA       23400.7
Gross :     59281.78      Net Pay :  57081.78
==========================================================================
```

==========PAYSLIP==========

Emp no e102 Emp name :jerold wil

Basicpay 34500.67 PF : 2000

DA : 13800.27 PT : 200

HRA : 3450.07

Gross : 51751.01 Net Pay : 49551.01

==========

==========PAYSLIP==========

Emp no e103 Emp name :ramachandr

Basicpay 44580.56 PF : 2000

DA : 20061.25 PT : 200

HRA 4458.06

Gross : 69099.87 Net Pay : 66899.87

==========

==========PAYSLIP==========

Emp no e104 Emp name :anas

Basicpay 15580.39 PF : 2000

DA : 5453.14 PT : 200

HRA 12464.31

Gross : 33497.84 Net Pay : 31297.84

==========

PL/SQL procedure successfully completed.

Program 5:

Given the following tables: ITEM_MASTER (item no, name, stock, unit price) [Apply the Primary key and check constraint for stock and price as >0] [Insert 5 records] ITEM_TRANS (item no, quantity and trans date) Create a package PCK_ITEM includes a function CHK_ITEM and a procedure PROC_ITEM. Function CHK_ITEM gets one argument item no and is used to check whether the parameter item no exists in ITEM_MASTER and should return 1 if exist. Otherwise, 0 and displays proper message. Procedure PROC_ITEM gets two arguments item no and quantity, and is used to perform the following if item exists. If required quantity is not available, give appropriate message. If available, insert a record of this transaction to ITEM_TRANS and modify the stock in ITEM_MASTER. Write a PL/SQL program to accept ITEM_NO and Quantity needed of required item. Use Package to do the transaction process (Transaction date can be current date). OUTPUT to be shown as follows:

```
Enter value for accept_itemno: 1
old   5:            X:=&accept_itemno;
new   5:            X:=1;
Enter value for quantity: 3
old   6:          M:=&quantity;
new   6:          M:=3;
Item :aa   Quantity  :3  Price  :15 Total Amount  :45
```

SQL> connect system/tiger

Connected.

SQL> create table item_master(

2 itemno number(5),

3 name varchar2(10),

4 stock number(5),

5 unit_price number(5),

6 constraint stock_price check(stock>0 and unit_price>0));

Table created.

```
SQL> create table item_trans(
  2  itemno number(10),
  3  qty number(5),
  4  tdate date);

Table created.

SQL> desc item_master;
 Name                          Null?    Type
 ----------------------------------------- -------- ----------------------------
 ITEMNO                                    NUMBER(5)
 NAME                                      VARCHAR2(10)
 STOCK                                     NUMBER(5)
 UNIT_PRICE                                NUMBER(5)

SQL> desc item_trans;
 Name                          Null?    Type
 ----------------------------------------- -------- ----------------------------
 ITEMNO                                    NUMBER(10)
 QTY                                       NUMBER(5)
 TDATE                                     DATE

SQL> insert into item_master values(101,'printer',800,200);
1 row created.
SQL> insert into item_master values(102,'keyboard',700,100);
1 row created.
SQL> insert into item_master values(103,'mouse',300,1500);
1 row created.
SQL> insert into item_master values(104,'pendrive',100,150);
1 row created.
```

SQL> select * from item_master;

ITEMNO	NAME	STOCK	UNIT_PRICE
101	printer	800	200
102	keyboard	700	100
103	mouse	300	1500
104	pendrive	100	150

SQL> ed pck_item.sql

```
create or replace package pck_item as
function ch_item( n number ) return number;
procedure proc_item(n number,q number);
end pck_item;
/
```

SQL> ed pck_body.sql

```
create or replace package body pck_item as
function ch_item(n in number) return number is
ino number(5);
begin
        select itemno into ino from item_master where itemno=n;
        return 1;
        exception when no_data_found then
        return 0;
end ;
procedure proc_item(n in number,q in number) is
ino number(5);
iname varchar(10);
stk number(10);
price number(5);
```

```
totamt number(10);
begin
        if(pck_item.ch_item(n)=1) then
                select itemno,name,stock,unit_price into ino,iname,stk,price from item_master
where itemno=n;
        if(stk<q) then
                dbms_output.put_line('required stock not available');
        else
        insert into item_trans values(ino,q,sysdate);
        update item_master set stock=stock-q where itemno=n;
        totamt:=q*price;
        dbms_output.put_line(RPAD('Item:',10)||RPAD(iname,10)||RPAD('Quantity:',10)||RP
AD(q,10)||RPAD('Price:',10)||RPAD(price,10)||RPAD('Total:',10)||RPAD(totamt,10));
end if;
end if;
end;
end pck_item;
/
```

SQL> ed prock_item.sql

```
set serveroutput on;
        prompt Enter Item Number:;
        accept ino;
        prompt Enter Quantity:;
        accept qty;
begin
if(pck_item.ch_item(&ino)=1) then
        dbms_output.put_line('Item Exists.');
else
        dbms_output.put_line('Item Does Not Exists.');
end if;
```

```
        pck_item.proc_item(&ino,&qty);
end;
/
```

OUTPUT:

```
SQL> set serveroutput ON;
Sql>@"C:\Users\divya\OneDrive\Pictures\Screenshots\pck_item.sql";
```

Package created.

```
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\pck_body.sql";
```

Package body created.

```
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\prock_item.sql";
Enter Item Number:
101
Enter Quantity:
5
old   2: if(pck_item.ch_item(&ino)=1) then
new   2: if(pck_item.ch_item(101)=1) then
old   7:       pck_item.proc_item(&ino,&qty);
new   7:       pck_item.proc_item(101,5);
```

Item Exists.

```
Item :    printer   Quantity :5        Price :   200       Total :   1000
PL/SQL procedure successfully completed.

SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\prock_item.sql";
Enter Item Number:
106
Enter Quantity:
4
old   2: if(pck_item.ch_item(&ino)=1) then
```

new 2: if(pck_item.ch_item(106)=1) then

old 7: pck_item.proc_item(&ino,&qty);

new 7: pck_item.proc_item(106,4);

Item Does Not Exists.

PL/SQL procedure successfully completed.

Program 6:

Create a package which includes a function to compute the factorial of a number, a procedure to compute the value of nCr, and another procedure to compute nPr both uses the factorial function. Execute the package program for the required calculation.

Pck_math.sql

```
CREATE OR REPLACE PACKAGE pck_math AS
   FUNCTION fact(n NUMBER) RETURN NUMBER;
   PROCEDURE nCr(n NUMBER, r NUMBER);
   PROCEDURE nPr(n NUMBER, r NUMBER);
END pck_math;
/
```

Create the Package Body

Pck_bodyy.sql

```
CREATE OR REPLACE PACKAGE BODY pck_math AS
   -- Function to compute factorial
   FUNCTION fact(n NUMBER) RETURN NUMBER IS
      res NUMBER := 1;
   BEGIN
      IF n < 0 THEN
         RETURN NULL; -- factorial not defined for negative numbers
      ELSIF n = 0 THEN
         RETURN 1;
      ELSE
         FOR i IN 1..n LOOP
            res := res * i;
         END LOOP;
         RETURN res;
```

```
    END IF;
  END fact;
  -- Procedure to compute nCr
  PROCEDURE nCr(n NUMBER, r NUMBER) IS
    result NUMBER;
  BEGIN
    IF r > n OR n < 0 OR r < 0 THEN
      DBMS_OUTPUT.PUT_LINE('Invalid inputs for nCr.');
    ELSE
      result := fact(n) / (fact(r) * fact(n - r));
      DBMS_OUTPUT.PUT_LINE('nCr(' || n || ', ' || r || ') = ' || result);
    END IF;
  END nCr;
  -- Procedure to compute nPr
  PROCEDURE nPr(n NUMBER, r NUMBER) IS
    result NUMBER;
  BEGIN
    IF r > n OR n < 0 OR r < 0 THEN
      DBMS_OUTPUT.PUT_LINE('Invalid inputs for nPr.');
    ELSE
      result := fact(n) / fact(n - r);
      DBMS_OUTPUT.PUT_LINE('nPr(' || n || ', ' || r || ') = ' || result);
    END IF;
  END nPr;
END pck_math;
/
```

OUTPUT:

```
SQL> connect system/tiger
Connected.
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\pck_math.sql";
Package created.
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\pck_bodyy.sql";
Package body created.
SQL> set serveroutput ON;
SQL> BEGIN
        pck_math.nCr(5, 2);
    END;
   /
nCr(5, 2) = 10
PL/SQL procedure successfully completed.


SQL> BEGIN
        pck_math.nPr(5, 2);
   END;
    /
nPr(5, 2) = 20
PL/SQL procedure successfully completed.
```

Program 7:

Create a trigger to update the MASTER table when a record is inserted into SALES table and create another trigger to update the MASTER table when a record is inserted or updated or deleted in NEWSTOCK table. Assume the suitable columns for all the tables.

SQL> create table MASTER(

```
  2  PID number(10)primary key,
  3  pname varchar(10)not null,
  4  total_stock number(7),
  5  total_sales number(9));
```

Table created.

SQL> desc MASTER;

Name	Null?	Type
PID	NOT NULL	NUMBER(10)
PNAME	NOT NULL	VARCHAR2(10)
TOTAL_STOCK		NUMBER(7)
TOTAL_SALES		NUMBER(9)

SQL> insert into MASTER values(101,'MOUSE',100,0);

1 row created.

SQL> insert into MASTER values(102,'DVD',70,0);

1 row created.

SQL> insert into MASTER values(103,'CD',90,0);

1 row created.

SQL> insert into MASTER values(104,'KEYBOARD',150,0);

1 row created.

```
SQL> insert into MASTER values(105,'MONITOR',50,0);

1 row created.


SQL> select*from MASTER;

       PID   PNAME      TOTAL_STOCK    TOTAL_SALES
----------  ---------- ----------- --------------- ------------------------

       101   MOUSE             100                0
       102   DVD               70                 0
       103   CD                90                 0
       104   KEYBOARD          150                0
       105    MONITOR          50                 0


SQL> create table SALES (
  2     SID varchar(10)primary key,
  3     PID number(10),
  4     sales_qty number(20));


Table created.

SQL> desc SALES;
 Name                          Null?                Type
SID                            NOT NULL             VARCHAR2(10)
 PID                                                 NUMBER(10)
 SALES_QTY                                           NUMBER(20)
SQL> create table NEWSTOCK (
  2     stock_id number(10) primary key,
  3      PID number(10),
  4      stock_qty number(10));


Table created.
```

```
SQL> desc NEWSTOCK;
 Name                      Null?          Type
STOCK_ID                     NOT NULL     NUMBER(10)
 PID                                       NUMBER(10)
 STOCK_QTY                                 NUMBER(10)
```

SQL> ed SALES.sql

```
CREATE OR REPLACE TRIGGER update_master_after_sales
after insert on SALES
for each row
begin
        update MASTER set total_stock=total_stock-:NEW.sales_qty,
        total_sales=total_sales+:NEW.sales_qty where PID=:NEW.PID;
End;
/
```

SQL> ed NEWSTOCK.sql

```
CREATE OR REPLACE TRIGGER update_master_after_newstock
after insert or update or delete on NEWSTOCK
for each row
begin
        IF inserting then
        update MASTER set total_stock=total_stock+:NEW.stock_qty where
 PID=:NEW.PID;
ELSIF updating then
        update MASTER set total_stock=total_stock+(:NEW.stock_qty-:OLD.stock_qty)
where PID=:NEW.PID;
ELSIF deleting  then
        update MASTER set total_stock=total_stock-:OLD.stock_qty where PID=:OLD.PID;
END IF;
```

```
END;
/


SQL> set serveroutput ON;
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\SALES.sql";
Trigger created.
SQL> insert into SALES values('S1',101,20);
1 row created.
SQL> insert into SALES values('S2',102,10);
1 row created.
SQL> insert into SALES values('S3',103,30);
1 row created.
SQL> select*from SALES;
SID         PID  SALES_QTY
---------- ---------- ----------
S1            101        20
S2            102        10
S3            103        30


 SQL> select*from MASTER;
     PID  PNAME      TOTAL_STOCK    TOTAL_SALES
---------- ---------- ----------- -------------  ---------------
     101   MOUSE             80                  20
     102   DVD               60                  10
     103   CD                60                  30
     104   KEYBOARD     150                   0
     105   MONITOR         50                  0
```

```
SQL> @"C:\Users\divya\OneDrive\Pictures\Saved Pictures\NEWSTOCK.sql";
```

Trigger created.

```
SQL> insert into NEWSTOCK values(201,101,60);

1 row created.

SQL> insert into NEWSTOCK values(202,102,40);

1 row created.

SQL> insert into NEWSTOCK values(203,103,55);

1 row created.

SQL> select*from NEWSTOCK;

  STOCK_ID        PID  STOCK_QTY
---------- ---------- ----------
       201        101         60
       202        102         40
       203        103         55


SQL> select*from MASTER;

       PID  PNAME      TOTAL_STOCK   TOTAL_SALES
---------- ---------- ----------  ------------------
       101   MOUSE           140            20
       102   DVD             100            10
       103   CD              115            30
       104   KEYBOARD        150             0
       105   MONITOR          50             0


SQL> update NEWSTOCK
  2     set stock_qty=80
  3     where stock_id=201;
1 row updated.
```

```
SQL> update NEWSTOCK
  2      set stock_qty=70
  3      where stock_id=202;
1 row updated.
SQL> select*from NEWSTOCK;
  STOCK_ID     PID  STOCK_QTY
---------- ---------- ----------
       201        101        80
       202        102        70
       203        103        55
SQL> select*from MASTER;
      PID   PNAME      TOTAL_STOCK    TOTAL_SALES
      101   MOUSE          160                20
      102   DVD            130                10
      103   CD             115                30
      104   KEYBOARD       150                 0
      105   MONITOR         50                 0


SQL> delete from NEWSTOCK where stock_id=201;
1 row deleted.
SQL> select*from MASTER;
      PID   PNAME     TOTAL_STOCK  TOTAL_SALES
      101    MOUSE          80                20
      102     DVD           130                10
      103    CD             115                30
      104    KEYBOARD       150                 0
      105     MONITOR        50                 0
```